AutoCloud-Agent

Multi-Agent Reinforcement Learning for Cloud Resource Management

GitHub Repository

K Chaitanya (25CS60R28)

Rahul K (25CS60R16)

Sri Lalitha Rama Siddesh Devarakonda (25CS60R80)

April 2026

Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur

Talk Outline

1. Problem & Motivation
2. System Architecture
3. Three RL Agents + I-PPO
4. Workload Forecaster
5. Safety Coordinator
6. Reward Design & Training
7. Results, Ablation & Stress Tests
8. Future Work & Conclusion

The Problem: Two Competing Objectives

Elasticity

Scale up *before* demand spikes
SLA: P95 latency $< 500\,\text{ms}$
VM boot delay: 30–90 s

vs.

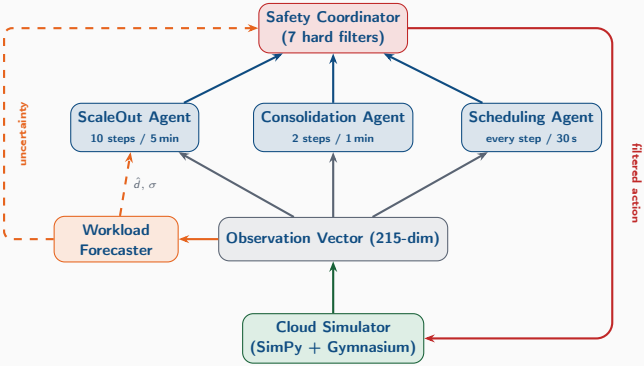
Cost Minimization

Drain idle VMs aggressively
Pack jobs tightly
Never pay for idle cores

Why existing methods fail:
Kubernetes HPA / AWS Auto Scaling are *reactive* — they respond *after* CPU crosses a threshold. By then, VMs need 60s to boot and SLA is already violated.

Why single-agent RL fails:
Joint action space $= 3 \times 2^{20} \times 5 \approx 15\,\text{M}$ combinations. Too large to learn efficiently from scratch.

System Architecture



Three Specialized Agents — Temporal Hierarchy

ScaleOut Agent

Action: $\{0, +1, +2\}$ VMs

Space: **3**

Every **10 steps** (5 min)

Capacity planning

Consolidation Agent

Action: drain mask $\{0, 1\}^{20}$

Space: 2^{20}

Every **2 steps** (1 min)

Cost optimization

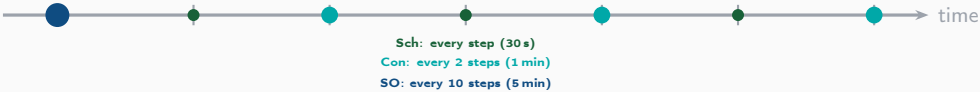
Scheduling Agent

Action: priority $\{0..4\}$

Space: **5**

Every **step** (30 s)

Job dispatch



Key: joint space $3 \times 2^{20} \times 5 \approx 15\text{M}$ $\rightarrow$ three independent spaces of size **3**, 2^{20}, **5**

Independent PPO (I-PPO) — How Each Agent Learns

PPO Clipped Objective

$$L^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t\hat{A}_t,\ \text{clip}(r_t,\ 1-\epsilon,\ 1+\epsilon)\hat{A}_t\right)\right]$$

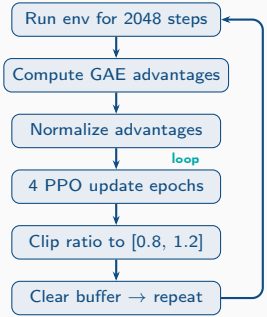
where $r_t = \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

Agent-Specific Reward Shaping:

ScaleOut: under/over-provisioning + running cost + proactive bonus

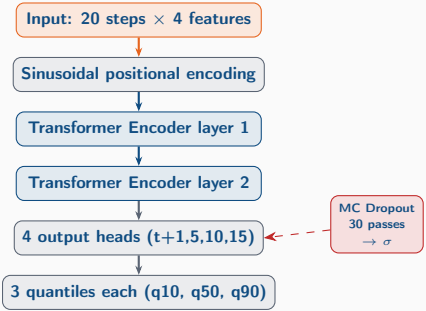
Consolidation: cost saving + SLA bonus − migration penalty

Scheduling: − mean wait time − SLA violations



Hyperparams: $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$, lr $= 3 \times 10^{-4}$, entropy $0.01 \rightarrow 0.001$

Workload Forecaster — Transformer + MC Dropout



Why MC Dropout?

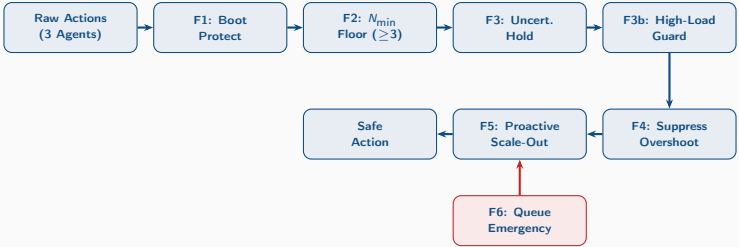
Keep dropout **on** at inference.
Run 30 forward passes → 30 different predictions.

$\sigma = \mathrm{var}(30 \text{ outputs})$

Low σ → model confident → act normally
High σ → model uncertain → Safety Coordinator blocks drains and forces scale-up

Forecaster output injected into obs[200:215]:
means $[\hat{d}_{t+1}, \hat{d}_{t+5}, \hat{d}_{t+10}, \hat{d}_{t+15}]$
sigmas $[\sigma_{t+1}, \sigma_{t+5}, \sigma_{t+10}, \sigma_{t+15}]$

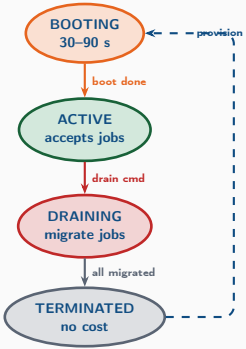
Safety Coordinator — 7-Filter Pipeline



Key insight: Hard constraints in the *reward* only penalise violations — they do not *prevent* them. The coordinator runs *after* the agents and *guarantees* no unsafe action ever reaches the simulator, regardless of how well (or poorly) the agents are trained.

Problem Modelling — State Space & VM Dynamics

VM State Machine



Formal Problem (Dec-POMDP)

State $s_t \in \mathbb{R}^{215}$:
Node features (CPU, mem, status), job features (wait, priority), global metrics + forecaster $\hat{d}, \sigma$

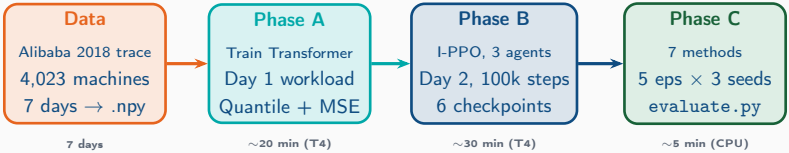
Actions (decomposed via I-PPO):
· ScaleOut: $\{0, +1, +2\}$ VMs
· Consolidation: drain mask $\{0, 1\}^{20}$
· Scheduling: priority strategy $\{0..4\}$

Transition: VM boot delays, job migration, stochastic workload arrival

Objective: Minimise $\sum_t \text{cost}_t$ subject to SLA $\geq 95\%$

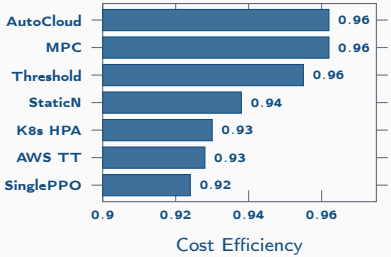
Joint space: $3 \times 2^{20} \times 5 \approx 15$M decomposed to 3 independent sub-spaces

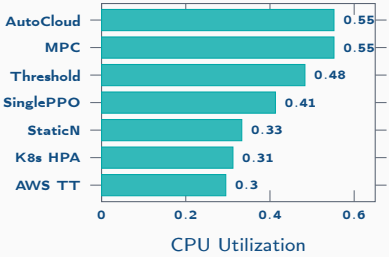
Training Pipeline



Experimental protocol: Each method evaluated over 5 episodes $\times$ 3 random seeds $=$ 15 runs on Alibaba Cluster Trace 2018 (4,023 machines, 7 days). Trained on Kaggle T4 GPU; total wall-clock time $\sim$50 min. All 7 methods share the same simulator and metrics.

Results — Alibaba Cluster Trace 2018

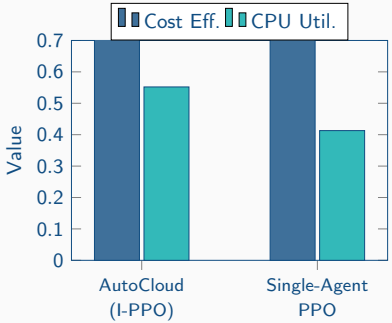




All 7 methods achieve 100% SLA compliance.
Differentiator is cost efficiency and CPU utilization.

AutoCloud beats K8s HPA by +3.4% cost, +77% CPU util.
Matches MPC while being fully *learned*, not hand-tuned.

Ablation — Why Multi-Agent Decomposition?



Why I-PPO wins

1. **Smaller action spaces**
   Each agent: 3, 2^{20}, or 5 — not 15M joint
2. **Temporal hierarchy**
   Fast decisions (30s) don't mix with slow (5min)
3. **Per-agent rewards**
   Clearer learning signal per subtask
4. **Credit assignment**
   Easier to debug — which agent caused a violation?

+4.1% cost efficiency **+33.7%** CPU util.
over single-agent RL

Stress Testing — 4 Out-of-Distribution Scenarios

(1) Ramp-Up
$1\times \rightarrow 3\times$ linear ramp
Tests provisioning

(2) Early Shock
$4\times$ spike at step 15
Filters 5+6 fire

(3) Choppy Plateau
$2.5 \times \pm$ noise
No VM thrashing

(4) Trough + Recovery
$0.3\times$ then $2.5\times$
Avoid over-drain

Why stress test? These patterns are *out-of-distribution* — the agents were trained on the Alibaba Day 2 trace. Stress tests verify the Safety Coordinator and learned policy generalise to unseen workload shapes.

Limitations & Future Directions

Current Limitations

1. **Homogeneous VMs**
   Real clouds offer heterogeneous instances with varying CPU/memory/price
2. **Single-cluster scope**
   Production systems span thousands of clusters across regions
3. **Simplified networking**
   No cross-rack latency or bandwidth modelling
4. **Single-day training**
   Multi-day training with domain randomisation would improve generalisation

Future Directions:

- Heterogeneous VMs with bin-packing strategies
- Multi-cluster federation (hierarchical agents)
- Vertical scaling (resize individual VMs)
- Transfer learning across traces (Azure, Borg)
- Spot/preemptible instance price-aware scaling
- Real Kubernetes deployment

Key open question:
Can temporal I-PPO decomposition scale to 100+ node types and multi-region federated clusters?

Conclusion

Three Contributions

1. Multi-Agent Decomposition (I-PPO)
3 agents at different temporal frequencies — 15M joint space → three manageable spaces

2. Uncertainty-Aware Forecasting
Transformer + MC Dropout → proactive scaling before the spike arrives

3. Safety Coordinator
7 hard-constraint filters guarantee no unsafe action ever reaches the simulator

96.2% cost efficiency
55.2% CPU utilization
100% SLA compliance

Matches **MPC** (best classical)
Beats **K8s HPA** by 3.4% cost
+4.1% cost, **+33.7%** CPU over single-agent RL

Refs: Schulman et al. (PPO, 2017)
Vaswani et al. (Transformers, 2017)
Gal & Ghahramani (MC Dropout, 2016)
Zhang et al. (AWARE, ATC 2023)